Claude Sonnet 5.5 on Jacobin G-function plan for java.time

This plan assumes "G-functions" means the Go implementations of Java methods. Dependencies are listed at the Java API level, plus the Go-side primitives each class needs. They have not been checked against the current G-function table, so treat the lists as a map of the JDK's internal references, not an exact call graph.

1. Dependency sublists

Common to all classes: `Object`, `String`, `Objects`, `Math` (`addExact`, `floorDiv`, `floorMod`), `DateTimeException`, `ArithmeticException`, `Comparable`, `Serializable`. These are omitted from the lists below.

java.time.DayOfWeek (enum)

- `Enum`, `TemporalAccessor`, `TemporalAdjuster`
- `ChronoField` (DAY_OF_WEEK), `ValueRange`, `TemporalQueries`, `TemporalQuery`
- `TextStyle`, `Locale`, `DateTimeFormatterBuilder` (only for `getDisplayName`)

java.time.Month (enum)

- Same as DayOfWeek, plus `ChronoField` (MONTH_OF_YEAR), `IsoChronology` (`isIsoLeap`), `Chronology`, `TemporalQueries.chronology()`

java.time.temporal.ChronoUnit (enum)

- `TemporalUnit`, `Duration` (each unit holds one), `Temporal` (`between` calls `until`), `Enum`

java.time.Duration

- `TemporalAmount`, `TemporalUnit`, `ChronoUnit`, `Temporal`
- `Instant`, `LocalTime` (for `between` and `toXxxPart` helpers), `ChronoField` (NANO_OF_SECOND)
- `BigInteger` / `BigDecimal` (`multipliedBy`, `dividedBy`, `parse`), `regex.Pattern` (`parse`)
- `UnsupportedTemporalTypeException`

java.time.Period

- `ChronoPeriod`, `TemporalAmount`, `ChronoUnit` (YEARS/MONTHS/DAYS), `TemporalUnit`
- `LocalDate`, `ChronoLocalDate`, `Temporal`, `Chronology` / `IsoChronology`, `TemporalQueries`
- `regex.Pattern` (`parse`), `ChronoField` (PROLEPTIC_MONTH, in `addTo`)

java.time.LocalDate

- `Temporal`, `TemporalAdjuster`, `ChronoLocalDate`
- `ChronoField`, `ChronoUnit`, `ValueRange`, `TemporalField`, `TemporalUnit`, `TemporalAmount`, `TemporalQueries`
- `IsoChronology`, `Year`, `Month`, `DayOfWeek`, `Period`
- `LocalTime`, `LocalDateTime`, `ZoneId`, `ZoneOffset`, `ZoneRules`, `ZonedDateTime` (`atStartOfDay(zone)`), `Instant`, `Clock`
- `DateTimeFormatter` (`parse`, `format`), `ChronoLocalDateTime`

java.time.LocalTime

- `Temporal`, `TemporalAdjuster`, `ChronoField`, `ChronoUnit`, `ValueRange`, `TemporalQueries`
- `LocalDate`, `LocalDateTime`, `Instant`, `Clock`, `ZoneId`, `ZoneOffset`
- `DateTimeFormatter`, `Duration` (`plus` / `minus` with a `TemporalAmount`)

java.time.LocalDateTime

- `Temporal`, `TemporalAdjuster`, `ChronoLocalDateTime`
- `LocalDate`, `LocalTime`, `ZoneOffset`, `ZoneId`, `ZonedDateTime`, `Instant`, `Clock`
- `ChronoField`, `ChronoUnit`, `TemporalAmount`, `Period`, `Duration`
- `DateTimeFormatter`, `IsoChronology`

java.time.ZoneId (abstract)

- `ZoneOffset`, `ZoneRegion` (package-private), `ZoneRules`, `ZoneRulesProvider`
- `TimeZone` (`systemDefault`), `SHORT_IDS` map, `Clock`
- `TextStyle`, `Locale`, `DateTimeFormatterBuilder` (`getDisplayName`), `TemporalAccessor`, `TemporalQueries`

- `ZoneRulesException`, `Set`, `Map`

java.time.ZoneOffset

- Extends `ZoneId`
- `TemporalAccessor`, `TemporalAdjuster`, `ChronoField` (OFFSET_SECONDS), `ZoneRules.of(offset)`
- `ConcurrentHashMap` cache, `LocalTime` (`parse`), `TemporalQueries`

java.time.ZonedDateTime

- `Temporal`, `ChronoZonedDateTime`
- `LocalDateTime`, `LocalDate`, `LocalTime`, `ZoneId`, `ZoneOffset`, `ZoneRules`, `ZoneOffsetTransition`, `Instant`, `Clock`
- `OffsetDateTime`, `Duration`, `Period`, `ChronoField`, `ChronoUnit`, `TemporalAmount`
- `DateTimeFormatter`, `TemporalQueries`

java.time.format.DateTimeFormatter

- `DateTimeFormatterBuilder` (plus its internal printer-parsers), `DateTimeParseException`
- `Locale`, `DecimalStyle`, `ResolverStyle`, `FormatStyle`, `TextStyle`, `SignStyle`
- `ParsePosition`, `java.text.Format`, `StringBuilder` / `Appendable`
- `ChronoField`, `IsoChronology`, `Chronology`, `TemporalAccessor`, `TemporalQueries`
- `ZoneId`, `ZoneOffset`, `Instant`
- Internal `Parsed` class and `DateTimeTextProvider` (locale-based month and day names)
- `Map`, `Set`

java.time.temporal.TemporalAdjusters

- `TemporalAdjuster` (lambdas), `Temporal`, `ChronoField`, `ChronoUnit`, `DayOfWeek`, `Objects`
- Several methods use `Temporal.with(...)` and `get(...)`, and a few assume ISO chronology

Suggested build order (bottom-up)

1. Exceptions, `ValueRange`, `ChronoField`, `ChronoUnit` (stub `Duration` fields first), `TemporalQueries`
2. `IsoChronology`, `DayOfWeek`, `Month`, `Year`
3. `LocalDate`, `LocalTime`, `LocalDateTime`, `Duration`, `Period`
4. `ZoneOffset`, `ZoneRules`, `ZoneId`, `Instant`, `ZonedDateTime`
5. `TemporalAdjusters`, `DateTimeFormatter` and its builder (largest piece; start with ISO constants and numeric patterns)

2. Where Go on Windows can't carry the load

Go's `time` package is mostly fine for calendar math (proleptic Gregorian, same as java.time's ISO). The problems are the OS and data dependencies.

Area	Affected classes/methods	Why it fails on Windows	Solution
IANA tz database	`ZoneId.of("America/Chicago")`, `ZoneRegion`, `ZonedDateTime.of/ofInstant/withZoneSameInstant`, `LocalDate.atStartOfDay(zone)`, `ZoneRules`	`time.LoadLocation` reads `/usr/share/zoneinfo` or `$GOROOT/lib/time/zoneinfo.zip`. Windows has no system zoneinfo, and end users have no GOROOT.	`import _ "time/tzdata"` or build with `-tags timetzdata` (~450 KB). For more control, `go:embed` your own `zoneinfo.zip` and load it with `time.LoadLocationFromTZData`.
System default zone	`ZoneId.systemDefault()`, `ZonedDateTime.now()`, `LocalDateTime.now()`, `Clock.systemDefaultZone()`	`time.Local` on Windows is built from `GetTimeZoneInformation` and its name is just "Local". Windows keys (e.g. "Central Standard Time") are not IANA IDs.	Call `GetDynamicTimeZoneInformation` via `golang.org/x/sys/windows`, read `TimeZoneKeyName`, and map it to IANA using an embedded CLDR `windowsZones` table. Honor `TZ` first, as the JDK does.
Enumerating zones	`ZoneId.getAvailableZoneIds()`, `ZoneRulesProvider`	Go exposes no API to list tz names, and the embedded `tzdata` doesn't either.	Generate a names list at build time and `go:embed` it (or enumerate your own embedded zip).
DST gaps and overlaps	`ZonedDateTime.of(LocalDateTime, zone)`, `withEarlierOffsetAtOverlap`, `ZoneOffsetTransition`	Go's `time.Date` explicitly doesn't guarantee which result you get in a gap or overlap. Java mandates shift-forward for gaps and earlier-offset for overlaps. Go also exposes no transition list.	Parse TZif yourself from the embedded zip, or probe candidate offsets with `t.In(loc).Zone()`. Implement the JDK resolution rules in your own `ZoneRules`. This is OS-independent but most visible on Windows because of the tzdata gap above.

Area	Affected classes/methods	Why it fails on Windows	Solution
Clock precision	`LocalTime.now()`, `LocalDateTime.now()`, `Instant.now()`, `Clock.systemUTC()`	The Windows wall clock often ticks at ~1–15.6 ms through `time.Now()`. Java on Windows returns sub-microsecond values.	Call `GetSystemTimePreciseAsFileTime` via `x/sys/windows` (or `syscall` lazy-proc) and convert from 100 ns FILETIME ticks (epoch 1601). Keep the monotonic reading separate for `nanoTime`.
Locale data	`DateTimeFormatter.ofPattern("EEEE, MMM")`, `ofLocalizedDate/Time`, `DayOfWeek/Month.getDisplayName`, `ZoneId.getDisplayName`, `DecimalStyle`	Go has no CLDR. Windows locale info is only reachable through `GetLocaleInfoEx` / `GetUserDefaultLocaleName`.	Embed a small English/ROOT table for names and patterns and add locales on demand. Optionally query `GetLocaleInfoEx` for the default locale. Document unsupported locales.
Nanosecond range	`Duration`, `ChronoUnit.between`, `Period` / `Duration` arithmetic	`time.Duration` is int64 ns (about ±292 years). Java's `Duration` is seconds (int64) + nanos (int).	Use your own `{seconds int64; nanos int32}` struct. Never wrap `time.Duration`. This is OS-independent.

Classes that don't need OS help: `DayOfWeek`, `Month`, `ChronoUnit`, `Period`, `TemporalAdjusters`, `ZoneOffset`, and the pure arithmetic in `LocalDate` / `LocalTime` / `LocalDateTime`. The one exception is the locale display-name methods above.

Recommended approach

Implement your own epoch-day/epoch-second arithmetic, copied from the JDK algorithms, for `LocalDate`, `LocalTime`, and `Duration`. Use Go's `time` only as an input source (current time, tz data), and put all zone logic behind one internal `zoneRules` abstraction. That keeps behavior identical across Linux, macOS, and Windows, and confines the Windows-specific code to three small files: `now_windows.go`, `syszone_windows.go`, and the embedded data.